ملاحظات الدرس — لك

طريق الحافلة

الحلول المثلى، المستوى ٣، الدرس ٣

المدة
٢٠ دقيقة

المواد
ورقة وقلم، مسافات تقريبية بين خمسة أماكن تعرفها، اختياري: هاتف عليه تطبيق «درسي» لتشغيل برنامجك

أهداف التعلّم

- تشرح لماذا ينفجر عدد الطرق.
- تكتب «أقرب جار» في بايثون وتحسّن طريقًا بطريقة 2-opt.
- تحكم على الطريق بطوله وبأطول رحلة فيه.

النشاط: رحلة ذهاب وإياب أعدل

١. الخريطة (٥ دقائق): ارسم خمسة أماكن نقاطًا، واكتب مسافة تقريبية بين كل زوج منها في جدول مثل d.
٢. البناء (٥ دقائق): من مكان البداية، اذهب دائمًا إلى أقرب مكان لم تزره، ثم عُد. ارسم الطريق واجمع طوله.
٣. التحسين (٥ دقائق): إن تقاطع طريقان، فاعكس المقطع الذي بينهما واجمع من جديد. وكرّر ما دام ذلك يفيد.
٤. الرحلة (٥ دقائق): لكل محطة، اجمع المسافة منها حتى النهاية. افعل ذلك في الاتجاهين، واختر الاتجاه الذي تكون فيه أطول رحلة أقصر.

يمكن أن تكون المسافات دقائق مشي. واستعمل أسماء خيالية إن فضّلت ألا تُظهر أماكن حقيقية.

للنقاش

- لماذا تنتهي «أقرب جار» كثيرًا بمرحلة طويلة؟
- كيف يمكن أن تتوقّف 2-opt قبل أفضل طريق؟
- هل يجوز أن تتخطّى الحافلة محطة لتوفير الوقود؟

تحقّق من نفسك

- ارسم طريقًا فيه تقاطع + الطريق بعد تبديل 2-opt واحد.

جرّبها هذا الأسبوع

ارسم خريطة لخمسة أماكن تزورها في الأسبوع (البيت، والمدرسة، والسوق، وبيت صديق، والعيادة) مع مسافات تقريبية. خطّط رحلة ذهاب وإياب من البيت بقاعدة «أقرب جار»، وارسمها، وابحث عن تقاطع: بدّله واجمع المسافة من جديد. ثم اسأل: لو انضمّ إليك صديق في أول محطة، فكم سيركب؟

ورقة عمل

طريق الحافلة

الحلول المثلى، المستوى ٣، الدرس ٣

الاسم | الفصل | التاريخ

١ طريقان لهما الطول نفسه. في أحدهما تركب قرية ٥٠ دقيقة، وفي الآخر لا يركب أحد أكثر من ٣٠. أيّهما تختار المدرسة؟

○ الذي لا يركب فيه أحد أكثر من ٣٠ دقيقة ○ أيّهما: فالكلفة واحدة ○ الذي يفضّله السائق

٢ على خريطة كل طرقها خطوط مستقيمة، حين يتقاطع طريقان في مسار الحافلة، يستطيع تبديل 2-opt دائمًا أن يجعل الطريق أقصر.

○ صح ○ خطأ

٣ ماذا يطبع هذا البرنامج؟

```
route = [0, 1, 2, 3, 4]
i, j = 1, 3
mid = list(reversed(route[i:j + 1]))
print(route[:i] + mid + route[j + 1:])
```

○ [0, 3, 2, 1, 4] ○ [4, 3, 2, 1, 0] ○ [0, 1, 3, 2, 4]

٤ d[a][b] هي المسافة بالكيلومتر بين المكانين a وb، و0 هو المدرسة. اكتب «أقرب جار»: أضف كل مرة أقرب مكان ليس في route بعد، حتى تدخل الأماكن الخمسة كلها. ويطبع البرنامج الطريق، وطوله (مع العودة)، والمسافة التي تركبها القرية الأولى.

```
d = [[0, 7, 1, 5, 2],
     [7, 0, 5, 2, 8],
     [1, 5, 0, 4, 3],
     [5, 2, 4, 0, 6],
     [2, 8, 3, 6, 0]]
route = [0]
# add the closest place not yet in route,
# until all 5 places are in

back = route[1:] + [0]
legs = zip(route, back)
km = sum(d[a][b] for a, b in legs)
print(route, km, km - d[0][route[1]])
```

اكتب برنامجك:

٥ رتّب الأسطر: دالة تُرجع طول الطريق، مع العودة إلى نقطة البداية.

رقّم الأسطر بالترتيب (١، ٢، ٣...)، وارسم أمام السطر سهمًا ← عن كل إزاحة يحتاج إليها. اشطب أي سطر لا ينتمي إلى البرنامج.

- [ ] `return km`
- [ ] `b = route[(i + 1) % len(route)]`
- [ ] `b = route[i + 1]`
- [ ] `for i in range(len(route)):`
- [ ] `def length(route):`
- [ ] `km += d[route[i]][b]`
- [ ] `km = 0`

✎ فكّر في طريق تسلكه كثيرًا. هل هو الأقصر، أم الأكثر أمانًا، أم الذي يناسب الآخرين؟ ما الذي ستغيّره، ومَن سيتأثر بذلك؟

طريق الحافلة

١ **الذي لا يركب فيه أحد أكثر من ٣٠ دقيقة**

حين تتساوى كلفة خطتين، يمكن لهدف ثانٍ أن يحسم بينهما.

٢ **صح**

استبدال الطريقين المتقاطعين بالطريقين اللذين لا يتقاطعان (بعكس المقطع الذي بينهما) أقصر دائمًا على خريطة مستوية، مثل قطرَي شكل رباعي مقارنةً بضلعين متقابلين من أضلاعه.

٣ **[0, 3, 2, 1, 4]**

route[1:4] هي [1, 2, 3]، ومعكوسها [3, 2, 1].

٤

```
d = [[0, 7, 1, 5, 2],
     [7, 0, 5, 2, 8],
     [1, 5, 0, 4, 3],
     [5, 2, 4, 0, 6],
     [2, 8, 3, 6, 0]]
route = [0]
while len(route) < 5:
    last = d[route[-1]]
    left = [v for v in range(5)
            if v not in route]
    near = min(left, key=lambda v: last[v])
    route.append(near)
back = route[1:] + [0]
legs = zip(route, back)
km = sum(d[a][b] for a, b in legs)
print(route, km, km - d[0][route[1]])
```

تمشي «أقرب جار» 0 ← 2 ← 4 ← 3 ← 1 ثم تعود: ١٩ كم، وتركب القرية الأولى (2) ١٨ كم منها لأنها تُؤخذ بعد ١ كم فقط.

٥ **من الأعلى إلى الأسفل: ٦، ٤، ✗، ٣، ١، ٥، ٢**

```
def length(route):
    km = 0
    for i in range(len(route)):
        b = route[(i + 1) % len(route)]
        km += d[route[i]][b]
    return km
```

يجعل (len(route % المكانَ التالي لآخر محطة هو المكانَ الأول، فتُحسب العودة.